Generalitat de Catalunya
Departament d'Educació
Institut Obert de Catalunya

Nom i cognoms

CFGS Desenvolupament d'Aplicacions Multiplataforma
0490 – Programació de serveis i processos
RA1, RA2

EAC1

(Curs 2026–27 / 1r semestre)

Presentació i resultats d'aprenentatge

Aquest exercici d'avaluació continuada (EAC) es correspon amb els continguts treballats a les unitats 1, **Programació multiprocés i paral·lela** i 2 **Programació multifil.**

Els **resultats d'aprenentatge** que es plantegen són:

- RA 1. Desenvolupa aplicacions compostes per diversos processos reconeixent i aplicant principis de programació paral·lela.
- RA 2. Desenvolupa aplicacions compostes per diversos fils d'execució analitzant i aplicant llibreries específiques del llenguatge de programació.

Criteris d'avaluació

La puntuació màxima assignada a cada activitat s'indica a l'enunciat.

Els criteris que es tindran en compte per avaluar el treball de l'alumnat són els següents:

- CA1.1. Reconeix les característiques de la programació concurrent i els seus àmbits d'aplicació.
- CA1.2. Identifica les diferències entre programació paral·lela i programació distribuïda, els seus avantatges i els seus inconvenients.
- CA1.3. Analitza les característiques dels processos i de la seva execució per part del sistema operatiu.
- CA1.4. Caracteritza els fils d'execució i en descriu la seva relació amb els processos.
- CA1.5. Utilitza classes per programar aplicacions que creen subprocessos.
- CA1.6. Utilitza mecanismes per compartir informació amb els subprocessos iniciats.
- CA1.7. Utilitza mecanismes per sincronitzar i obtenir el valor retornat pels subprocessos iniciats.
- CA1.8. Desenvolupa aplicacions que gestionin i utilitzin processos per a l'execució de diverses tasques en paral·lel.
- CA1.9. Depura i documenta les aplicacions desenvolupades.
- CA2.1. Identifica situacions en què sigui útil l'ús de diversos fils en un programa.
- CA2.2. Reconeix els mecanismes per crear, iniciar i finalitzar fils.
- CA2.3. Programa aplicacions que implementin diversos fils.

- CA2.4. Identifica els possibles estats d'execució d'un fil i programa aplicacions que els gestionin.
- CA2.5. Utilitza mecanismes per compartir informació entre diversos fils d'un mateix procés.
- CA2.6. Desenvolupa programes formats per diversos fils sincronitzats mitjançant tècniques específiques.
- CA2.7. Estableix i controla la prioritat de cadascun dels fils d'execució.
- CA2.8. Depura i documenta els programes desenvolupats.
- CA2.9. Analitza el context d'execució dels fils.
- CA2.10. Analitza llibreries específiques del llenguatge de programació que permeten la programació multifil.
- CA2.11. Reconeix els problemes derivats de la compartició d'informació entre els fils d'un mateix procés.

Forma i data de lliurament

Per respondre les qüestions plantejades, heu de completar el projecte Java proporcionat i el fitxer de respostes teoricopràctiques.

Un cop finalitzat l'exercici d'avaluació continuada heu d'enviar el document des de l'apartat **0490 Lliurament EAC1** de l'aula, dins del termini establert. Tingueu en compte que el sistema no permetrà fer lliuraments després de la data i hora indicades.

El fitxer lliurat serà un únic fitxer comprimit en format .zip i inclourà el fitxer de respostes teoricopràctiques i el projecte Java proporcionat completat:

- El fitxer **respostes_teoriques.md**, amb les respostes teoricopràctiques dels exercicis 1, 2, 3 i 4.
- El projecte Java proporcionat, amb els punts marcats amb **TODO** completats. Cal reanomenar la carpeta del projecte com **DAM_0490_EAC1_Cognom1_InicialDelCognom2**.

El termini de lliurament finalitza a les **23:55 h** del dia **20/10/2026.** La proposta de solució de l'EAC es publicarà el dia 16/11/2026 i les qualificacions el dia 16/11/2026.

Nom i cognom

Enunciat

Context

En un mòdul de programació, el professorat rep una carpeta amb diversos lliuraments d'alumnes. Cada lliurament s'ha de validar per comprovar si conté els fitxers esperats, si es pot processar correctament i quin resultat dona.

Per automatitzar aquesta feina, partireu d'un projecte Java ja creat. No heu de dissenyar una aplicació des de zero: heu de llegir, entendre, modificar i completar els punts indicats.

El sistema té dues fases:

1. Una primera versió basada en **processos externs**, on el programa principal executa un validador com a procés independent.
2. Una segona versió basada en **fils**, on diversos lliuraments es processen concurrentment dins del mateix procés Java.

Condicions generals

- Cal programar en **Java**.
- Al projecte de les preguntes 1, 2, 3 i 4, cal mantenir els **noms de paquets, classes i mètodes** indicats.
- Al projecte de les preguntes 1, 2, 3 i 4, no s'han de canviar les **signatures públiques** dels mètodes proporcionats.
- No s'han de canviar els **formats de sortida** indicats a l'enunciat.
- Tots els **mètodes públics** que reben paràmetres han de validar les entrades nules, buides o invàlides segons correspongui.
- Al projecte de les preguntes 1, 2, 3 i 4, cal completar només els punts marcats amb TODO.
- Cal evitar que el programa quedi penjat indefinidament.
- Cal tancar correctament els recursos utilitzats.
- Els **criteris de correcció** de les quatre primeres preguntes dependran del **funcionament del codi** i de les sortides indicades.
- Les preguntes teoricopràctiques dels exercicis 1, 2, 3 i 4 s'han de respondre al fitxer respostes_teoriques.md. Les preguntes de l'exercici 5 us serveixen per justificar i revisar el disseny que incorporeu al mateix projecte Java proporcionat.

Projecte base

Se us proporciona un projecte Java amb diverses classes ja creades. Algunes les haureu de completar i d'altres només us serveixen de suport. Respecteu els noms de paquets, classes i mètodes, ja que formen part de l'estructura de l'activitat i permeten verificar-ne el funcionament.

Classes del projecte:

Classe	Funció dins del projecte
Lliurament	Representa un lliurament detectat. Desa l'identificador de l'alumne, la ruta de la carpeta del lliurament i el temps simulat de processament. No decideix si el lliurament és correcte o no. **No cal modificar-la**.
CarregadorLliuraments	Llegeix la carpeta lliuraments/, detecta quins subdirectoris hi ha i construeix una llista d'objectes Lliurament. No crea els fitxers ni decideix el resultat. **No cal modificar-la**.
CorrectorLliurament	Conté la lògica comuna per validar un lliurament: localitzar el fitxer, llegir-ne el contingut i decidir si és correcte. **Cal completar-la** a l'exercici 1.
ValidadorLliurament	Simula una eina externa de validació. S'executa com a programa independent i reutilitza CorrectorLliurament. **No cal modificar-la**.
CorrectorProcessos	Programa principal de la fase amb processos. Executa diversos ValidadorLliurament amb ProcessBuilder, gestiona processos en paral·lel i recull resultats. **Cal completar-la** a l'exercici 2.
ResultatCorreccio	Objecte de dades que representa el resultat d'una correcció feta amb fils: alumne, resultat, temps, missatge i fil que l'ha processat. **No cal modificar-la**.
TascaCorreccio	Tasca que processa un lliurament dins d'un fil. Reutilitza CorrectorLliurament i retorna un ResultatCorreccio. **No cal modificar-la**.
MarcadorCorreccions	Objecte compartit per totes les tasques. Acumula totals de lliuraments processats, correctes, errors i temps de processament. **Cal completar-la** a l'exercici 4.
CorrectorExecutor	Programa principal de la fase amb fils. Crea el pool, envia tasques, recull resultats, tanca l'executor i genera l'informe final. **Cal completar-la** a l'exercici 3.

Estructura corregible

Aquesta EAC està pensada perquè les preguntes 1, 2, 3 i 4 es puguin corregir de manera **senzilla i automàtica**. Per això, en el projecte base no heu de crear classes noves ni decidir lliurement l'arquitectura.

Cada exercici indica uns **mètodes concrets** que heu de completar. El professorat podrà comprovar el funcionament executant les classes principals i, si cal, provant directament aquests mètodes.

És especialment important que:

- no canvieu els noms de les classes,
- no canvieu els noms dels mètodes indicats,
- no canvieu els tipus dels paràmetres ni els tipus de retorn,
- no canvieu els codis de sortida,
- no canvieu les línies principals de sortida que s'indiquen com a esperades,
- valideu sempre les **precondicions** dels mètodes públics amb paràmetres.

Font de dades

El projecte base inclou aquesta carpeta de dades:

lliuraments/

Dins d'aquesta carpeta hi ha diversos subdirectoris, un per cada alumne simulat:

```
lliuraments/
 alumne01/
   lliurament.txt
 alumne02/
   lliurament.txt
 alumne03/
   lliurament.txt
 ...
```

Cada subdirectori representa el lliurament d'un alumne. Dins de cada subdirectori hi ha un fitxer lliurament.txt que conté una paraula que simula el resultat del lliurament:

- correcte: el lliurament s'ha de considerar vàlid.
- error: el lliurament s'ha de considerar incorrecte.

La **font de veritat** és sempre el fitxer:

lliuraments/alumneXX/lliurament.txt

No hi ha cap camp dins de Lliurament ni dins de CarregadorLliuraments que digui si un lliurament és correcte o incorrecte. El resultat s'ha de descobrir llegint el fitxer.

Flux general

Abans de validar res, el programa ha de descobrir quins lliuraments existeixen. Aquesta feina la fa CarregadorLliuraments:

CarregadorLliuraments.carregar()

-> llegeix la carpeta lliuraments/

-> detecta els subdirectoris alumne01, alumne02, ...

-> crea un objecte Lliurament per cada carpeta trobada

-> retorna List<Lliurament>

Cada objecte Lliurament conté:

- l'identificador de l'alumne,
- la ruta de la carpeta del lliurament,
- un temps simulat de processament amb variació pseudoaleatòria, que serveix perquè algunes tasques triguin més que altres.

La lògica de validació està centralitzada en CorrectorLliurament: CorrectorLliurament.validar(ruta)

-> localitza la ruta del fitxer lliurament.txt

-> llegeix i normalitza el contingut

-> retorna true si el contingut és correcte

-> retorna false si el lliurament no és validable

Flux de la fase amb **processos**:

CarregadorLliuraments.carregar()

-> retorna List<Lliurament>

-> CorrectorProcessos recorre la llista

-> per cada Lliurament llança un procés ValidadorLliurament

-> ValidadorLliurament reutilitza CorrectorLliurament

-> el procés fill acaba amb codi 0 o 2

-> CorrectorProcessos recull el codi i genera el resum

Flux de la fase amb **fils**:

CarregadorLliuraments.carregar()

-> retorna List<Lliurament>

-> CorrectorExecutor crea un ExecutorService

-> envia una TascaCorreccio per cada Lliurament

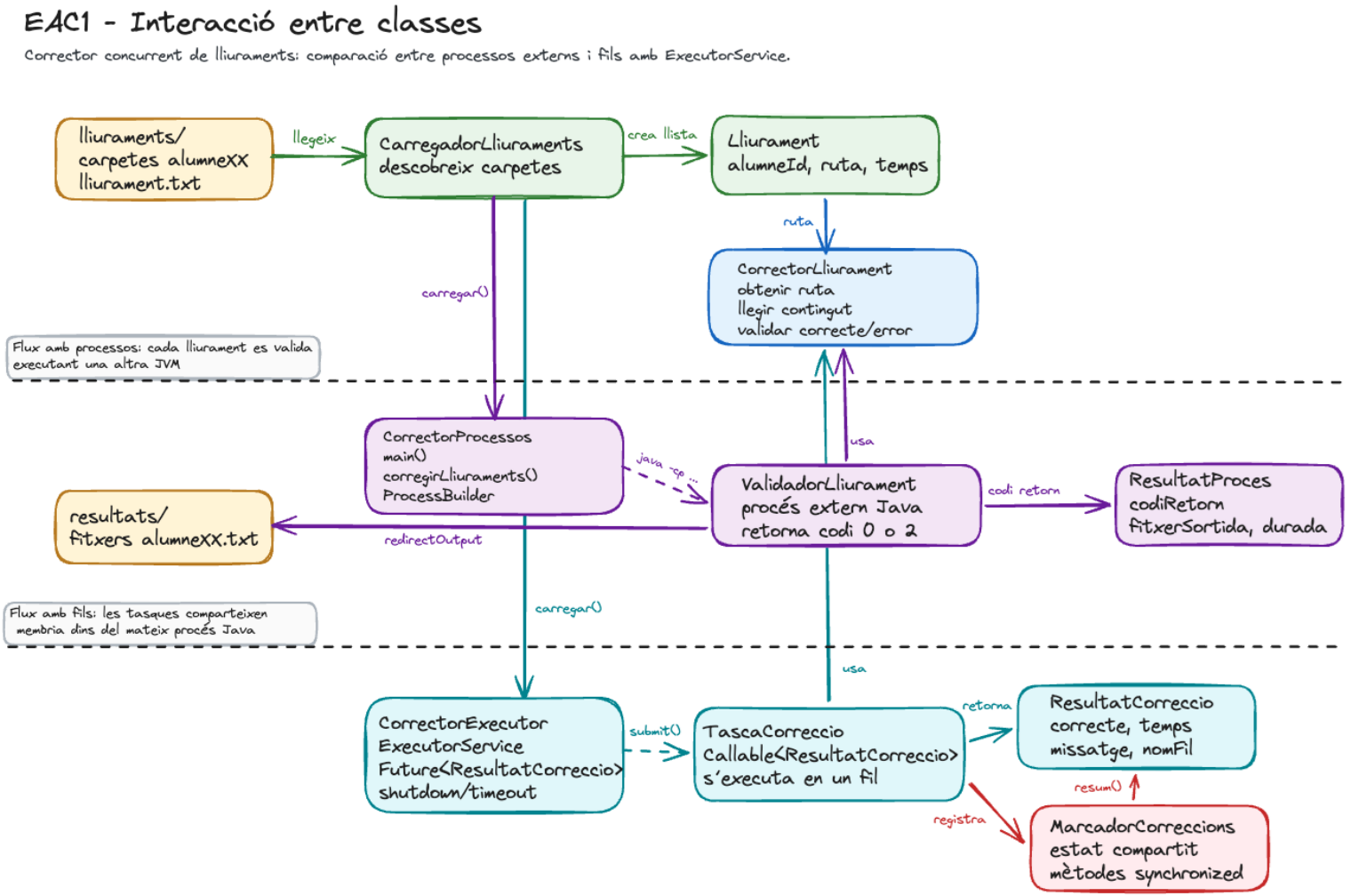
-> cada TascaCorreccio reutilitza CorrectorLliurament

-> cada TascaCorreccio retorna un ResultatCorreccio

-> cada TascaCorreccio actualitza MarcadorCorreccions

-> CorrectorExecutor recull els resultats i mostra l'informe final

La diferència principal és que a la fase amb processos la validació es fa en un **procés Java independent**, mentre que a la fase amb fils la validació es fa dins del **mateix procés Java**, repartida entre diversos fils.


EAC1 - Interacció entre classes
Corrector concurrent de lliuraments: comparació entre processos externs i fils amb ExecutorService.
lliuraments/
carpetes alumneXX
lliurament.txt
llegeix
CarregadorLliuraments
descobreix carpetes
crea llista
Lliurament
alumneId, ruta, temps
ruta
CorrectorLliurament
obtenir ruta
llegir contingut
validar correcte/error
carregar()
Flux amb processos: cada lliurament es valida executant una altra JVM
CorrectorProcessos
main()
corregirLliuraments()
ProcessBuilder
java -cp ...
usa
ValidadorLliurament
procés extern Java
retorna codi 0 o 2
codi retorn
ResultatProces
codiRetorn
fitxerSortida, durada
resultats/
fitxers alumneXX.txt
redirectOutput
Flux amb fils: les tasques comparteixen memòria dins del mateix procés Java
carregar()
usa
CorrectorExecutor
ExecutorService
Future<ResultatCorreccio>
shutdown/timeout
submit()
TascaCorreccio
Callable<ResultatCorreccio>
s'executa en un fil
retorna
ResultatCorreccio
correcte, temps
missatge, nomFil
resum()
registra
MarcadorCorreccions
estat compartit
mètodes synchronized

Exercici 1 - Lògica comuna de correcció

En aquest exercici heu de completar la classe:

```
dam.psp.eac1.CorrectorLliurament
```

CorrectorLliurament és la classe que conté la dinàmica comuna de correcció. Aquesta classe serà reutilitzada tant per ValidadorLliurament com per TascaCorreccio.

Heu de completar aquests mètodes, sense canviar-ne la signatura:

Mètode	**Què ha de fer**
obtenirRutaFitxerLliurament(Path ruta)	Ha de validar que ruta no sigui null. Si ruta és un directori, ha de retornar ruta.resolve("lliurament.txt"). Si ruta és un fitxer regular, ha de retornar la mateixa ruta. Si no és ni un directori ni un fitxer regular, ha de llançar una IllegalArgumentException.
llegirContingut(Path fitxer)	Ha de validar que fitxer no sigui null. Ha de llegir el contingut del fitxer, eliminar espais inicials i finals, i convertir-lo a minúscules. Si el fitxer no existeix, no és un fitxer regular o no es pot llegir, ha de llançar una IOException.
esLliuramentCorrecte(String contingut)	Ha de validar que contingut no sigui null. Ha de retornar true només si el contingut normalitzat és exactament correcte. En qualsevol altre cas ha de retornar false.

El mètode validar(Path ruta) ja està preparat per utilitzar aquests tres mètodes. Si la ruta no és vàlida o el fitxer no es pot llegir, ha de retornar false.

ValidadorLliurament ja utilitza CorrectorLliurament.

Per comprovar aquest exercici, executeu:

```
java -cp target/classes dam.psp.eac1.ValidadorLliurament alumne01
lliuraments/alumne01
```

El programa ha de mantenir aquests **codis de sortida**:

Situació	**Codi**
Lliurament correcte	0
Lliurament amb error o no validable	2

Situació	Codi
Arguments incorrectes	10

Sortida esperada per a alumne01:

```
ALUMNE=alumne01
RUTA=lliuraments/alumne01
RESULTAT=CORRECTE
```

Preguntes per a la vostra **reflexió personal**:

- Per què és útil separar la lògica de correcció en una classe independent?
- Per què ValidadorLliurament necessita un mètode main(String[] args)?
- Per què és útil retornar codis de sortida diferents?

Exercici 2 - Correcció amb processos externs

En aquest exercici heu de completar la classe:

```
dam.psp.eac1.CorrectorProcessos
```

CorrectorProcessos ha d'executar ValidadorLliurament com a **procés extern** utilitzant ProcessBuilder.

CorrectorProcessos no rep arguments de línia de comandes en aquesta EAC. Quan s'executa, sempre carrega tots els lliuraments de la carpeta lliuraments/, els corregeix i mostra el resum final.

En aquest exercici no n'hi ha prou amb processar els lliuraments de manera completament seqüencial. El programa ha de seguir dues fases:

1. **Llançament**: iniciar un procés fill per cada lliurament.
2. **Recollida**: quan tots els processos ja s'han iniciat, esperar-ne la finalització i construir els resultats.

Això permet que diversos processos fills estiguin en execució alhora.

Heu de completar aquests mètodes, sense canviar-ne la signatura:

Mètode	Què ha de fer
construirOrdreValidador(String alumneId, Path ruta)	Ha de validar que alumneId no sigui null ni buit, i que ruta no sigui null. Ha de construir la llista d'arguments necessària per executar dam.psp.eac1.ValidadorLliurament amb el Java actual, el classpath actual, l'identificador de l'alumne i la ruta del lliurament.
prepararProces(List<String> ordre, Path fitxerSortida)	Ha de validar que ordre no sigui null ni buida, que no contingui valors null o buits, i que fitxerSortida no sigui null. Ha de crear un ProcessBuilder, redirigir la sortida estàndard al fitxer indicat i unir la sortida d'error amb la sortida estàndard.

Mètode	Què ha de fer
esperarProces(Process proces)	Ha de validar que proces no sigui null. Ha d'esperar com a màxim TEMPS_MAXIM_PROCES_SEGONS. Si el procés acaba dins del temps, ha de retornar-ne el codi de sortida. Si no acaba dins del temps, ha de forçar-ne el tancament i retornar CODI_TIMEOUT.
corregirLliuraments(List<Lliurament> lliuraments)	Ha de validar que la llista no sigui null ni contingui valors null. Ha de crear el directori resultats/, iniciar tots els processos fills, desar-ne la informació necessària i, després, esperar-los per construir la llista de resultats. No ha d'esperar cada procés immediatament després de llançar-lo.

No heu de modificar ResultatProces ni mostrarResum(...). Són peces de suport per mantenir un format estable.

El projecte ja inclou aquestes constants:

```
CODI_TIMEOUT
TEMPS_MAXIM_PROCES_SEGONS
```

Feu-les servir per evitar que un procés fill pugui quedar penjat indefinidament.

Quan executeu:

```
java -cp target/classes dam.psp.eac1.CorrectorProcessos
```

La sortida final ha de ser:

```
TOTAL_PROCESSOS=10
CORRECTES=8
ERRORS=2
```

També s'han de generar fitxers a la carpeta resultats/, un per cada alumne.

Preguntes per a la vostra **reflexió personal**:

- Quin paper fa CorrectorProcessos com a **procés pare**?
- Quin paper fa ValidadorLliurament com a **procés fill**?
- Què podria passar si el procés pare no esperés la finalització del procés fill?
- Quina diferència hi ha entre llançar tots els processos abans de recollir resultats i executar-los un per un de manera seqüencial?

Exercici 3 - Correcció amb ExecutorService

En aquest exercici comenceu la fase amb **fils**. La validació del contingut ja no s'ha de tornar a implementar: TascaCorreccio ja reutilitza CorrectorLliurament.

Heu de completar la classe:

```
dam.psp.eac1.CorrectorExecutor
```

Heu de completar aquests mètodes, sense canviar-ne la signatura:

Mètode	Què ha de fer
enviarTasques(List<Lliurament> lliuraments, MarcadorCorreccions marcador, ExecutorService executor)	Ha de validar tots els paràmetres. Ha d'enviar una TascaCorreccio al pool per cada Lliurament i retornar la llista de Future<ResultatCorreccio>.
recollirResultats(List<Future<ResultatCorreccio>> futures)	Ha de validar que la llista no sigui null ni contingui valors null. Ha de recórrer els Future, obtenir cada resultat amb un temps màxim, cancel·lar les tasques que excedeixin el temps i retornar només els resultats obtinguts correctament.
tancarExecutor(ExecutorService executor)	Ha de validar que executor no sigui null. Ha de demanar el tancament ordenat, esperar un temps màxim i forçar el tancament si encara hi ha tasques pendents.

El projecte ja inclou aquestes constants:

```
TEMPS_MAXIM_TASCA_SEGONS
TEMPS_MAXIM_TANCAMENT_SEGONS
```

Feu-les servir per evitar valors màgics repetits al codi.

Quan executeu:

```
java -cp target/classes dam.psp.eac1.CorrectorExecutor 3
```

El programa ha de mostrar un informe final de l'execució. Els valors de temps poden variar, però l'estructura ha de ser semblant a:

```
InformeExecucio{midaPool=3, tempsRealMs=...,
marcador=MarcadorCorreccions{totalProcessats=10, totalCorrectes=8, totalErrors=2,
tempsTotalProcessamentMs=...}, resultats=10}
```

Internament, cada ResultatCorreccio ha de conservar el nom del fil que ha processat el lliurament. Aquest nom es pot consultar a través de getNomFil(), però no cal imprimir una línia per cada alumne en l'execució principal.

Prova d'estrès abans de sincronitzar

Quan hàgiu acabat l'exercici 3, el programa ja ha de saber treballar amb un ExecutorService, enviar tasques i recollir resultats. Ara bé, encara no heu resolt el problema de l'**estat compartit** de MarcadorCorreccions.

Per veure aquest problema, feu una prova d'estrès sense modificar el projecte. Executeu jshell indicant el classpath del projecte compilat:

```
jshell --class-path target/classes
```

Dins de jshell, enganxeu aquest codi:

```
import dam.psp.eac1.*;
import java.util.concurrent.*;
int total = 100000;
MarcadorCorreccions marcador = new MarcadorCorreccions();
ExecutorService executor = Executors.newFixedThreadPool(10);
for (int i = 0; i < total; i++) {
    executor.submit(() -> {
        ResultatCorreccio resultat = new ResultatCorreccio(
            "alumne-prova",
            true,
            1L,
            "Lliurament correcte",
            Thread.currentThread().getName()
        );
        marcador.registrar(resultat);
    });
}
executor.shutdown();
executor.awaitTermination(30, TimeUnit.SECONDS);
System.out.println("ESPERAT=" + total);
System.out.println("TOTAL_PROCESSATS=" + marcador.getTotalProcessats());
System.out.println("TOTAL_CORRECTES=" + marcador.getTotalCorrectes());
System.out.println("TOTAL_ERRORS=" + marcador.getTotalErrors());
System.out.println("COHERENT=" + (marcador.getTotalProcessats() == total
    && marcador.getTotalCorrectes() == total
    && marcador.getTotalErrors() == 0));
```

En aquest punt, és possible que el resultat **no sigui coherent**:

```
ESPERAT=100000
TOTAL_PROCESSATS=98742
TOTAL_CORRECTES=98691
TOTAL_ERRORS=0
COHERENT=false
```

Els números exactes poden variar entre execucions. Si en una execució surt coherent, repetiu la prova o augmenteu el valor de total. El més important és entendre que, sense protegir el marcador, el programa pot semblar correcte però no és fiable: diversos fils poden modificar els mateixos comptadors alhora i perdre actualitzacions.

Preguntes per a la vostra **reflexió personal**:

- Què aporta ExecutorService respecte de crear fils manualment?
- Per què fem servir Callable<ResultatCorreccio> i no només Runnable?
- Per què és important tancar correctament un ExecutorService?
- Quina diferència hi ha entre el temps acumulat de processament i el temps real d'execució?
- Per què la prova d'estrès pot mostrar totals incorrectes encara que totes les tasques hagin acabat?

Exercici 4 - Marcador compartit i sincronització

En aquest exercici heu de completar la classe:

```
dam.psp.eac1.MarcadorCorreccions
```

MarcadorCorreccions és compartida per totes les tasques. Per tant, és una zona on poden aparèixer problemes de concurrència.

Heu de protegir aquests mètodes perquè l'estat compartit sigui coherent:

Mètode	**Què ha de garantir**
registrar(ResultatCorreccio resultat)	Ha de validar que resultat no sigui null i actualitzar tots els comptadors com una operació segura.
getTotalProcessats()	Ha de llegir un valor coherent.
getTotalCorrectes()	Ha de llegir un valor coherent.
getTotalErrors()	Ha de llegir un valor coherent.
getTempsTotalProcessamentMs()	Ha de llegir un valor coherent.
resum()	Ha de construir el resum amb dades coherents.

Podeu resoldre-ho amb synchronized o amb un mecanisme equivalent que protegeixi correctament la secció crítica.

L'informe final s'ha de construir quan ja no quedin tasques pendents que puguin modificar el marcador. Si es mostra massa aviat, pot contenir dades incompletes.

La sortida final de CorrectorExecutor ha d'incloure:

- total de lliuraments processats,
- total de correctes,
- total d'errors,
- temps total acumulat de processament,
- temps real d'execució del programa,
- mida del pool utilitzat.

Per comprovar que la sincronització funciona, torneu a executar la mateixa prova d'estrès de l'exercici 3. Ara el resultat ha de ser coherent:

```
ESPERAT=100000
TOTAL_PROCESSATS=100000
TOTAL_CORRECTES=100000
TOTAL_ERRORS=0
COHERENT=true
```

Si el resultat continua sent incoherent, reviseu que totes les operacions que llegeixen o modifiquen l'estat compartit estiguin protegides amb el mateix mecanisme de sincronització.

Preguntes per a la vostra **reflexió personal**:

- Què és una **condició de cursa**?
- Per què pot aparèixer una condició de cursa en aquest exercici?
- Per què volatile no seria suficient per protegir increments de comptadors?
- Com podem saber que l'informe final és coherent?

Exercici 5 - Sistema concurrent propi

En aquest exercici heu de dissenyar i implementar un **sistema concurrent propi** utilitzant fils. La temàtica és lliure, però el sistema ha de tenir una estructura mínima comuna perquè es pugui revisar amb criteris clars.

Alguns exemples possibles:

- processament de comandes d'una botiga,
- validació de reserves d'un hotel,
- atenció de pacients en un centre mèdic,
- gestió de paquets en una empresa de repartiment,
- revisió de sol·licituds administratives,
- simulació de caixes d'un supermercat,
- processament d'imatges o fitxers.

No podeu copiar directament la temàtica del corrector de lliuraments canviant només els noms de les classes. Heu de proposar un context diferent i aplicar-hi els mateixos conceptes de concurrència.

El vostre sistema ha d'incloure, com a mínim:

- una implementació incorporada dins del mateix projecte Java proporcionat,
- una classe principal executable amb main(String[] args),
- una o més **classes de dades** que representin els elements que s'han de processar,
- una classe que representi una **tasca concurrent** amb Runnable o Callable,
- un ExecutorService amb un **pool de fils**,
- un mínim de **100 elements** de prova generats pel mateix programa,
- un pool amb un mínim de **4 fils**,
- enviament de múltiples tasques al pool,
- recollida o gestió dels resultats,
- un recurs o objecte d'**estat compartit**,
- un mecanisme de **sincronització** que eviti col·lisions sobre aquest estat compartit,
- tancament correcte de l'ExecutorService,
- una sortida final que permeti comprovar que el sistema ha processat correctament tots els elements.

La condició de concurrència obligatòria és aquesta: diverses tasques han de modificar el mateix **estat compartit**. Per tant, la sincronització ha de ser necessària, no només decorativa.

La sortida final ha de ser clara i ha d'incloure algun tipus de comprovació de coherència. Per exemple:

```
TOTAL_ELEMENTS=1000
TOTAL_PROCESSATS=1000
TOTAL_OK=934
TOTAL_ERROR=66
FILS_UTILITZATS=4
```

```
COHERENT=true
```

Podeu adaptar els noms de les línies al vostre problema, però s'ha de poder entendre fàcilment:

- quants elements s'havien de processar,
- quants s'han processat realment,
- quin resultat final s'ha obtingut,
- quants fils diferents han participat en el processament,
- si les dades finals són coherents.

Preguntes per a la vostra **reflexió personal**:

- Quin element del vostre sistema representa el treball que cal processar?
- Quina part del vostre sistema s'executa concurrentment?
- Quin estat compartit heu hagut de protegir?
- Com podeu demostrar que la sincronització funciona correctament?

Lliurament

Heu de lliurar:

- un únic fitxer comprimit en format .zip.

Aquest .zip ha de contenir el fitxer de respostes teoricopràctiques i el projecte Java proporcionat completat:

- el fitxer respostes_teoriques.md, amb les respostes dels exercicis 1, 2, 3 i 4,
- el projecte Java proporcionat, amb tots els exercicis completats.

Rúbrica de correcció

La qualificació de l'EAC és de **10 punts**.

Apartat	Puntuació	Criteri principal
Exercici 1	1,5 punts	Funcionament correcte de CorrectorLliurament i de la validació comuna del lliurament.
Exercici 2	1,5 punts	Funcionament correcte de CorrectorProcessos, ProcessBuilder, processos fills, codis de retorn i fitxers de sortida.
Exercici 3	1,5 punts	Funcionament correcte de CorrectorExecutor, enviament de tasques, ús de Future, recollida de resultats i tancament del pool.
Exercici 4	1,5 punts	Funcionament correcte de MarcadorCorreccions amb estat compartit protegit i resultats coherents a la prova d'estrès.
Exercici 5	4 punts	Disseny i implementació d'un sistema concurrent propi.

Els exercicis 1, 2, 3 i 4 es corregiran de manera **objectiva** a partir del codi, l'execució del programa, els codis de sortida, els fitxers generats i la coherència dels resultats.

La puntuació de l'exercici 5 es reparteix així:

Criteri de l'exercici 5	Puntuació
Defineix una temàtica pròpia i modela correctament els elements que s'han de processar.	0,75 punts
Utilitza correctament un ExecutorService amb un **pool de fils**.	0,75 punts
Defineix tasques concurrents amb Runnable o Callable.	0,75 punts
Gestiona correctament la finalització de les tasques, la recollida de resultats i el tancament del pool.	0,75 punts
Inclou un **estat compartit** i aplica un mecanisme de **sincronització** correcte.	0,75 punts
Mostra una sortida final clara, coherent i verificable.	0,25 punts

Recomanacions

- Al projecte de les preguntes 1, 2, 3 i 4, completeu només els punts marcats amb TODO.
- Comproveu primer que el projecte compila amb mvn clean compile.
- No deixeu fils o processos en execució després d'acabar el programa.
- Mostreu missatges d'execució clars, amb identificador d'alumne i nom del fil o procés.
- Vigileu especialment els recursos compartits.
- Si una execució concurrent dona resultats diferents cada vegada, reviseu la sincronització.